

Desenvolvimento Web 2

Aula 6

Processamento Assíncrono em JavaScript

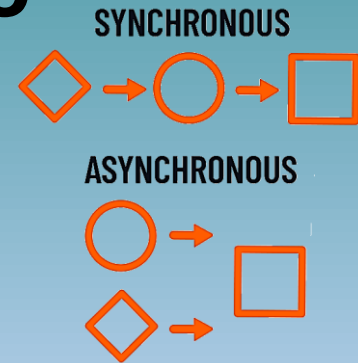
l.bertholdo@ifsp.edu.br

Conteúdo

- Processamento Síncrono e Assíncrono
- Sequência das Funções
- Callbacks
 - Callbacks Assíncronos
- Promises
 - Métodos then, catch e finally

Processamento Síncrono e Assíncrono

- **Processamento síncrono** é aquele em que as instruções do código são executadas exatamente na ordem em que foram escritas.
- Dessa forma, a execução de uma linha de código fica bloqueada até que o procedimento da linha anterior tenha terminado de executar.
- No **processamento assíncrono** esse bloqueio não existe. Se uma instrução é muito demorada, a instrução seguinte pode ter sua execução iniciada ainda que a anterior não tenha acabado.
- Embora a linguagem JavaScript trabalhe de modo síncrono por padrão, ela fornece diversos recursos para implementar o processamento assíncrono, entre eles: as **funções *callback assíncronas*** e o **objeto Promise**.



Sequência das Funções

- Em JavaScript, assim como em outras linguagens de programação, por padrão, as funções são executadas **na sequência em que são chamadas**, não na sequência em que são definidas.

```
<body>
  <p id="mensagem"></p>
  <script>
    function exhibe(texto) {
      let msg = document.getElementById("mensagem");
      msg.innerHTML = msg.innerHTML + texto + "<br>";
    }
    function funcao1() {
      exhibe("Olá!");
    }
    function funcao2() {
      exhibe("Tchau!");
    }
    funcao1();
    funcao2();
  </script>
</body>
```

Esse exemplo retorna o texto:

"Olá!"

"Tchau!"

Mas se as chamadas das duas funções fossem trocadas de ordem, o retorno seria:

"Tchau!"

"Olá!"

Sequência das Funções

- Ao escrever o código, procuramos **controlar a sequência** em que as funções são executadas. Um exemplo simples seria uma sequência lógica para um programa que faz adição de dois números: primeiro o programa faz o **cálculo** e depois exibe o **resultado**:

```
<p id="resultado"></p>
<script>
function exibe(result) {
  document.getElementById("resultado").innerHTML = result;
}
function calcula(num1, num2) {
  let soma = num1 + num2;
  return soma;
}
let result = calcula(3, 8);
exibe(result);
</script>
```

Chamamos a função **calcula()**, guardamos o resultado e então chamamos a função **exibe()**.

```
<p id="resultado"></p>
<script>
function exibe(result) {
  document.getElementById("resultado").innerHTML = result;
}
function calcula(num1, num2) {
  let soma = num1 + num2;
  exibe(soma);
}
calcula(3, 8);
</script>
```

Ou podemos chamar a função **calcula()** e deixar que ela própria chame a função **exibe()**.

Callbacks

- Uma outra forma de controlar a execução das funções seria por meio de um **callback**. De forma resumida, **callback** é uma função passada como argumento para outra função.

```
<p id="resultado"></p>
<script>
function exhibe(result) {
  document.getElementById("resultado").innerHTML = result;
}
function calcula(num1, num2, funcaoCallback) {
  let soma = num1 + num2;
  funcaoCallback(soma);
}
calcula(3, 8, exhibe);
</script>
```

exibe é passado como uma função *callback* para **calcula()**. Na sequência, o código de **calcula()** é executado, incluindo a chamada de **funcaoCallback()**, o que resulta na execução da função **exibe()**. Com isso, o **resultado** é exibido somente após o **cálculo** ser concluído. **Obs.:** Ao passar o nome de uma função como argumento, não se deve usar parênteses.

*Esse código exemplifica a sintaxe das funções callback. Porém, esse tipo de função passa a ser útil de fato quando precisamos executar **instruções assíncronas**, ou seja, em situações em que uma função precisa ser executada antes do término da tarefa de outra função (por exemplo, o carregamento de um arquivo muito grande).*

Callbacks Assíncronos

- Uma forma simples de demonstrar um **callback assíncrono** é usar a função JavaScript **setTimeout()**, especificando um *callback* a ser executado após um determinado tempo.

```
<p>A página será atualizada em 2 segundos...</p>
<h1 id="mensagem"></h1>
<script>
  function exibeMensagem() {
    document.getElementById("mensagem").innerHTML = "Olá!";
  }
  setTimeout(exibeMensagem, 2000);
</script>
```

A página será atualizada em 2 segundos...

Olá!

Nesse exemplo, **exibeMensagem** é passado como uma função *callback* para a função **setTimeout()**, cujo código é executado internamente (incluindo a chamada da função **exibeMensagem()**). Com isso, a mensagem é exibida somente após dois segundos (2000 milissegundos). Aqui, podemos dizer que a função **exibeMensagem()** é executada de forma assíncrona, porque qualquer função que fosse chamada depois do **setTimeout()** seria executada antes de **exibeMensagem()**.

Callbacks Assíncronos

- No exemplo abaixo, o assincronismo fica claro quando vemos a função **exibeAviso()** ser executada antes da função **exibeMensagem()**.

```
<h1 id="mensagem"></h1>
<script>
  function exibeAviso() {
    document.getElementById("mensagem").innerHTML = "Vou te cumprimentar!";
  }
  function exibeMensagem() {
    document.getElementById("mensagem").innerHTML = "Olá!";
  }
  setTimeout(exibeMensagem, 2000);
  exibeAviso();
</script>
```

Devido ao *delay* da função **setTimeout()**, **exibeMensagem()** é executada de forma **assíncrona** (fora de ordem), ou seja, é executada depois da função **exibeAviso()**, mesmo tendo sido invocada antes.

Vou te cumprimentar!

Olá!

Embora **callbacks assíncronos** sejam úteis, eles podem se tornar difíceis de escrever e de depurar. Podemos ter um callback dentro de outro callback, que está em outro callback, e assim por diante. Essa situação, conhecida como **“inferno de callbacks” (callback hell)**, gera um **encadeamento de código** que não é apenas confuso, mas também propenso a erros. Por conta disso, atualmente, costuma-se usar um método mais moderno de programação assíncrona baseado no uso de **promises**.

Promises

- A ES6 criou um objeto denominado **Promise** destinado a manipular um valor que pode estar disponível no futuro, ou seja, uma “promessa” de existência futura que poderá ou não se concretizar.
- Um *promise* produz um **valor** após uma operação assíncrona ser concluída com êxito, ou um **erro** se ela não for concluída, seja porque excedeu um limite de tempo, por um erro de rede, ou outro motivo.
- As operações bem-sucedidas são indicadas pela chamada da função **resolve** e os erros pela chamada da função **reject**. A sintaxe para instanciar uma *promise* é a seguinte:

Ao declarar uma *promise*, a **função construtora** (`new Promise`) recebe como argumento uma **função executora** (`function (resolve, reject)`).

```
let promise = new Promise(function(resolve, reject) {  
    // Faz uma chamada assíncrona e conclui ou rejeita esta chamada  
});
```

Promises

- A **função executora** recebe dois argumentos: as funções *callback* **resolve** e **reject**. A chamada dessas funções é feita dentro da **função executora**, que é executada automaticamente quando **new Promise** é chamado.
- A função executora da *promise* deve chamar uma das funções *callback*: **resolve** ou **reject**.
- Como a função executora precisa lidar com operações assíncronas, o objeto *promise* retornado por **new Promise** contém a informação do **estado** da execução: iniciada, concluída ou rejeitada.

```
let promise = new Promise(function(resolve, reject) {  
    if ( /* condição */ ) { resolve("Ok!"); }  
    else { reject("Erro!"); }  
});
```

Promises

- Um objeto *promise* possui as propriedades internas **state** e **result**, que podem ter os seguintes valores:

state:

- pending**: quando a função executora foi iniciada.
- fulfilled**: quando a *promise* foi concluída.
- rejected**: quando a *promise* foi rejeitada.

result:

- undefined**: quando o valor de **state** é "pending".
- value**: quando a função **resolve(value)** é chamada.
- error**: quando a função **reject(error)** é chamada.

*As propriedades **state** e **result** são inacessíveis no código, mas elas podem ser inspecionadas, ou seja, podemos verificar seus valores usando a ferramenta de depuração do navegador.*

```
6   function exhibe(texto) {
7       document.getElementById("mensagem").innerHTML = texto;
8   }
9   let promise = new Promise(function(resolve, reject) {
10       let x = 0;
11       if (x == 0) {
12           resolve("Ok!");
13       } else {
14           reject("Erro!");
15       }
16   });
17   promise.then(
18       function(value) {exibe(value);},
19       function(error) {exibe(error);}
20   );
```

The screenshot shows the browser's developer console with the 'Scope' tab selected. Under the 'Script' section, the 'promise' object is expanded, showing its internal properties. Two red arrows point to the '[[PromiseState]]' and '[[PromiseResult]]' properties, which are 'fulfilled' and 'Ok!' respectively. The console also shows the 'Global' and 'Call Stack' sections.

```
▼ Scope
▼ Script
  ▼ promise: Promise
    ▼ [[Prototype]]: Promise
      ► catch: f catch()
      ► constructor: f Promise()
      ► finally: f finally()
      ► then: f then()
      Symbol(Symbol.toStringTag):
      ► [[Prototype]]: Object
    ► [[PromiseState]]: "fulfilled"
    ► [[PromiseResult]]: "Ok!"
    ► Global
  ▼ Call Stack
    ► (anonymous)
```

Promises

- Após criar uma *promise*, é preciso definir uma **função consumidora**. Essa função usa o resultado da *promise*, e é notificada quando a **função executora** conclui ou rejeita a operação.
- Para criar um vínculo entre a **função executora** e a **função consumidora**, são usados os métodos **then()**, **catch()** e **finally()** do objeto **Promise**.
- Esse vínculo é necessário para que as duas funções estejam sincronizadas quando a *promise* for concluída (resolve) ou rejeitada (reject).

```
let promise = new Promise(function(resolve, reject) {  
    resolve("Ok!");  
});  
const consumer = () => {  
    promise.then(  
        value => { console.log(value); },  
        error => { console.log(error); }  
    )  
}
```

← Função *callback* da função executora

← Função consumidora

Método then

- Na **função consumidora**, o objeto **Promise** deve chamar o método **then()**, o qual aceita duas funções como parâmetros: uma para lidar com um resultado (resolve) e outra para lidar com um erro (reject).

```
promise.then(  
  value => { console.log(value); },  
  error => { console.log(error); }  
)
```

- Se apenas o resultado bem-sucedido for necessário, pode-se passar apenas o argumento do **resultado**. Se o interesse for apenas no erro, pode-se passar **null** no argumento do **resultado**.

```
promise.then(  
  value => { console.log(value); }  
)
```

```
promise.then(  
  null,  
  error => { console.log(error); }  
)
```

Método then

- O exemplo a seguir usa o método **then()** para tratar resultados e erros de uma requisição assíncrona.
- Para isso, será usada a **PokeAPI** (<https://pokeapi.co/>) que fornecerá informações sobre Pokémons e será usada como base para concluir ou rejeitar a requisição usando *promises*.
- Primeiramente, será criada uma função genérica que recebe uma URL da PokeAPI como argumento e retorna uma *promise*.
- Se a chamada à API for bem-sucedida, uma *promise* será retornada. Caso contrário, a *promise* é rejeitada.

Método then

- Função genérica para obter uma *promise*:

XMLHttpRequest: Objeto usado para realizar operações assíncronas. Por exemplo: atualizar uma página sem recarregá-la, solicitar e receber dados de um servidor após o carregamento da página, enviar dados para um servidor em segundo plano.

```
function getPromise(URL) {  
  let promise = new Promise(function (resolve, reject) {  
    let req = new XMLHttpRequest();  
    req.open("GET", URL);  
    req.onload = function () {  
      if (req.status == 200) { resolve(req.response); }  
      else { reject("Erro!"); }  
    };  
    req.send();  
  });  
  return promise;  
}
```

open: Conecta com o servidor da API e informa o método de envio de dados para o servidor (GET).

onload: Define a função a ser chamada quando a requisição receber uma resposta do servidor.

status: Status da resposta do servidor (200 = OK)

send: Envia a requisição para o servidor.

Método then

- Exemplo 1: Obtendo informações de cinco Pokémons:

```
let promise = getPromise('https://pokeapi.co/api/v2/pokemon?limit=5');
const consumer = () => {
  promise.then(
    value => {
      // Como a URL é válida, retornará cinco Pokémons no console do navegador.
      console.log({value});
    },
    error => {
      console.log({error});
    }
  );
};
consumer();
```

```
{
  "count": 1302,
  "next": "https://pokeapi.co/api/v2/pokemon?offset=5&limit=5",
  "previous": null,
  "results": [
    {
      "name": "bulbasaur",
      "url": "https://pokeapi.co/api/v2/pokemon/1/"
    },
    {
      "name": "ivysaur",
      "url": "https://pokeapi.co/api/v2/pokemon/2/"
    },
    {
      "name": "venusaur",
      "url": "https://pokeapi.co/api/v2/pokemon/3/"
    },
    {
      "name": "charmander",
      "url": "https://pokeapi.co/api/v2/pokemon/4/"
    },
    {
      "name": "charmeleon",
      "url": "https://pokeapi.co/api/v2/pokemon/5/"
    }
  ]
}
```

Elements Console Sources >> ⚙️ ⋮ ✕

top 🔍 Filter Default levels ▼

No Issues ⚙️

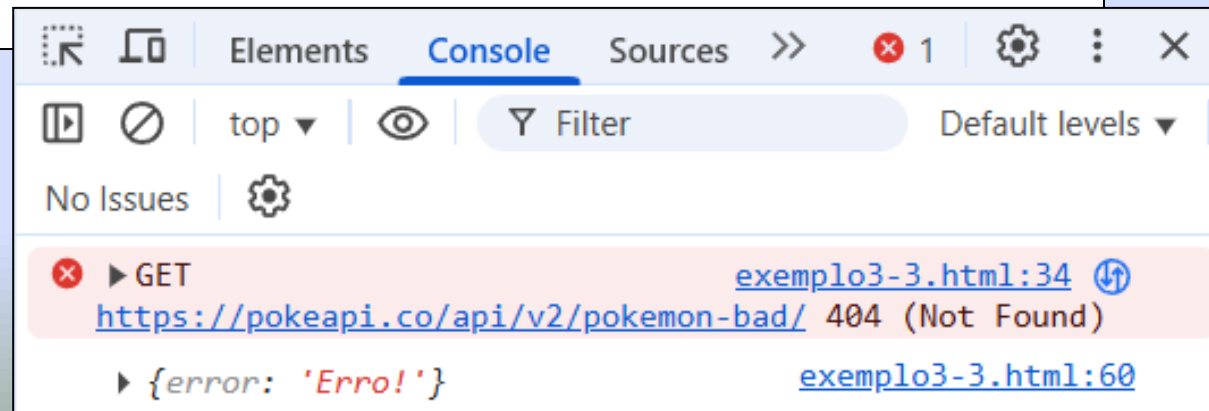
[exemplo3-3.html:44](#)

```
{value: '{"count":1302,"next":"https://pokeapi.co/api/v2/po...", "url": "https://pokeapi.co/api/v2/pokemon/5/"}'}
```


Método then

- **Exemplo 2:** Obtendo informações de pokémons a partir de uma URL inválida:

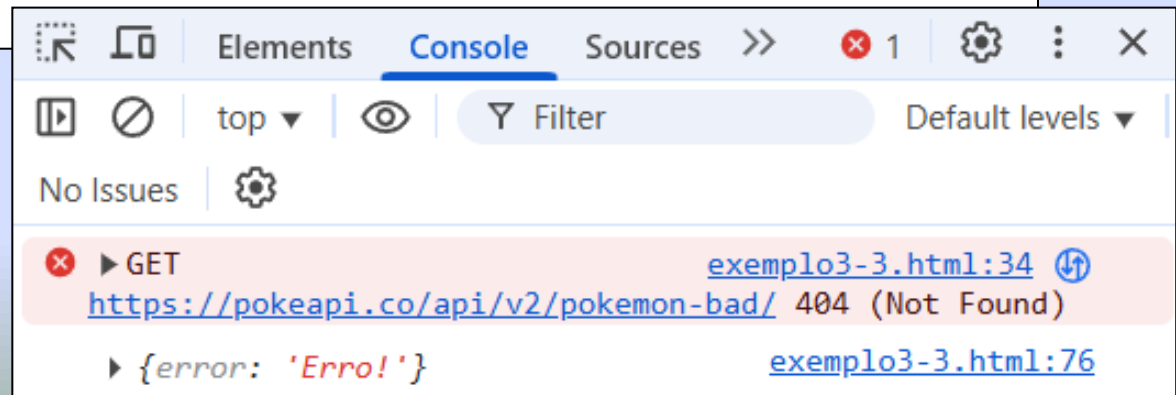
```
let promise = getPromise('https://pokeapi.co/api/v2/pokemon-bad/');
const consumer = () => {
  promise.then(
    value => {
      console.log({value});
    },
    error => {
      // Como a URL é inválida, retornará um erro no console do navegador.
      console.log({error});
    }
  );
};
consumer();
```



Método catch

- Uma forma melhor de tratar os erros de uma *promise*, é usando o método **catch()** em conjunto com o método **then()**.

```
let promise = getPromise('https://pokeapi.co/api/v2/pokemon-bad/');
const consumer = () => {
  promise.then(
    value => {
      console.log({value});
    }
  ).catch(error => {
    // Como a URL é inválida, retornará um erro no console do navegador.
    console.log({error});
  });
}
consumer();
```



Método finally

- O método **finally()** é usado para realizar tarefas adicionais como finalizar uma carga de dados, fechar uma conexão de rede etc. É chamado independentemente da *promise* retornar um resultado ou um erro.

```
let carregamento = true;
let promise = getPromise('https://pokeapi.co/api/v2/pokemon?limit=5');
promise.then(value => {
  console.log({value});
}).catch(error => {
  console.log(error)
}).finally(() => {
  carregamento = false;
  console.log(`Carregamento: ${carregamento}`);
});
```

 `{value: '{"count":1302,"next":"https://pokeapi.co/api/v2/po...", "url": "https://pokeapi.co/api/v2/pokemon/5/"}'}`

[exemplo3-3.html:86](#)

Carregamento: false

[exemplo3-3.html:91](#)

```
let carregamento = true;
let promise = getPromise('https://pokeapi.co/api/v2/pokemon-bad');
promise.then(value => {
  console.log({value});
}).catch(error => {
  console.log(error)
}).finally(() => {
  carregamento = false;
  console.log(`Carregamento: ${carregamento}`);
});
```

 `GET https://pokeapi.co/api/v2/pokemon-bad 404 (Not Found)`

[exemplo3-3.html:34](#)

Erro!

[exemplo3-3.html:99](#)

Carregamento: false

[exemplo3-3.html:102](#)

Referências

- MORAES, William Bruno. **Construindo aplicações com NodeJS**. São Paulo: Editora Novatec, 2023.
- SILVA, Maurício S. **JavaScript Essencial**. São Paulo: Editora Novatec, 2024.